CHƯƠNG V. CHẤM ĐIỂM, TRỌNG SỐ THUẬT NGỮ VÀ MÔ HÌNH KHÔNG GIAN VEVTOR

Truy xuất thông tin

ThS. Phạm Đức Cường
cuongpd@ptit.edu.vn

Posts and Telecommunications
Institute of Technology

Học viện Công nghệ Bưu chính Viễn thông

Nội dung

1. Các hàm chấm điểm
2. Trọng số thuật ngữ
3. TF-IDF (Term Frequency – Inverse Document Frequency)
4. Mô hình không gian vector

Truy xuất có xếp hạng

- Cho đến nay, các truy vấn của chúng ta đều là **Boolean**..
  - Tài liệu **hoặc là khớp hoặc không khớp**.
- Tốt cho **người dùng chuyên gia** có hiểu biết chính xác về nhu cầu của họ và tập tài liệu.
  - Cũng tốt cho **các ứng dụng**: Ứng dụng có thể dễ dàng xử lý **hàng nghìn kết quả**.
- Không tốt cho **đa số người dùng**.
  - Hầu hết người dùng **không có khả năng viết truy vấn Boolean** (hoặc có thể, nhưng họ nghĩ rằng làm vậy quá tốn công).
  - Hầu hết người dùng **không muốn phải xem qua hàng nghìn kết quả**.
    - Điều này **đặc biệt đúng đối với tìm kiếm trên web**.

Vấn đề với tìm kiếm Boolean: Feast or Famine

- Các truy vấn Boolean thường dẫn đến:
  - **quá ít kết quả (thậm chí = 0)**
  - **hoặc quá nhiều kết quả (hàng nghìn kết quả)**.
- **Ví dụ:**
  - Truy vấn 1:
    - "standard user dlink 650" → **hơn 200.000 kết quả**
  - Truy vấn 2:
    - "standard user dlink 650 no card found" → **0 kết quả**
- Cần **kỹ năng khá cao** để tạo ra một truy vấn cho số kết quả **vừa phải và dễ xử lý**.
- Toán tử Boolean gây ra vấn đề:
  - **AND** → thường cho **quá ít kết quả**
  - **OR** → thường cho **quá nhiều kết quả**

Mô hình truy xuất có xếp hạng

- Thay vì trả về **một tập hợp các tài liệu thỏa mãn biểu thức truy vấn**, trong **truy xuất có xếp hạng (ranked retrieval)**, hệ thống sẽ **trả về một thứ tự sắp xếp của các tài liệu (thường là các tài liệu đứng đầu)** trong bộ sưu tập đối với truy vấn đó
- **Truy vấn dạng văn bản tự do:**
  - Thay vì sử dụng **ngôn ngữ truy vấn với các toán tử và biểu thức**, truy vấn của người dùng chỉ đơn giản là **một hoặc nhiều từ trong ngôn ngữ tự nhiên**
- Về nguyên tắc, **đây là hai lựa chọn riêng biệt**. Tuy nhiên trong thực tế, **truy xuất có xếp hạng thường đi kèm với truy vấn văn bản tự do**, và **ngược lại**.

Feast or Famine: không còn là vấn đề trong truy hồi có xếp hạng

- Khi hệ thống tạo ra **một tập kết quả đã được xếp hạng**, thì **việc có quá nhiều kết quả không còn là vấn đề**.
  - Thực tế, **kích thước của tập kết quả không còn quan trọng**.
  - Hệ thống chỉ cần **hiển thị top k kết quả** (thường khoảng **10 kết quả đầu tiên**).
  - Nhờ vậy **không làm người dùng bị quá tải thông tin**.
- **Giả định quan trọng:** Thuật toán **xếp hạng hoạt động tốt**.

Chấm điểm là nền tảng của truy hồi có xếp hạng

- Chúng ta muốn **trả về các tài liệu theo thứ tự**, sao cho **những tài liệu có khả năng hữu ích nhất đối với người tìm kiếm** được đưa lên trước.
- Làm thế nào để chúng ta **xếp hạng các tài liệu trong bộ sưu tập** đối với một truy vấn?
- **Gán một điểm số (score)** – ví dụ trong khoảng **[0, 1]** – cho **mỗi tài liệu**.
- Điểm số này **đo lường mức độ "khớp" giữa tài liệu và truy vấn**.

Cách tiếp cận 1: Hệ số Jaccard

- Một **thước đo phổ biến để đo mức độ chồng lấp (overlap)** của hai tập hợp **A và B**.
- jaccard$(A,B) = |A \cap B| \,/\, |A \cup B|$
- jaccard$(A,A) = 1$
- jaccard$(A,B) = 0$ if $A \cap B = 0$
- A và B không nhất thiết phải **có cùng kích thước**.
- Giá trị luôn **nằm trong khoảng từ 0 đến 1**.

Hệ số Jaccard: Ví dụ tính điểm

- **Điểm khớp giữa truy vấn và tài liệu** mà **hệ số Jaccard** tính cho **mỗi tài liệu** dưới đây là bao nhiêu?
- **Truy vấn:** ides of march
- **Tài liệu 1:** caesar died in march
- **Tài liệu 2:** the long march

Các vấn đề khi dùng Jaccard để tính điểm

- **Nó không xem xét tần suất của từ (term frequency) -** một từ xuất hiện **bao nhiêu lần trong tài liệu**.
- **Các từ hiếm trong tập tài liệu mang nhiều thông tin hơn các từ xuất hiện thường xuyên.** Jaccard **không xem xét thông tin này**.
- **Chúng ta cần một cách tinh vi hơn để chuẩn hóa theo độ dài của tài liệu**.

Điểm số khớp giữa truy vấn và tài liệu

- Chúng ta cần **một cách để gán điểm số cho một cặp truy vấn / tài liệu**.
- Hãy bắt đầu với **một truy vấn chỉ có một từ (one-term query)**.
- Nếu **từ truy vấn không xuất hiện trong tài liệu**: **điểm số phải bằng 0**.
- **Từ truy vấn xuất hiện càng nhiều lần trong tài liệu**, thì **điểm số càng cao (nên như vậy)**.
- Chúng ta sẽ **xem xét một số phương án khác nhau cho việc này**.

Nhắc lại: Ma trận xuất hiện thuật ngữ–tài liệu dạng nhị phân

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	1	1	0	0	0	1
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0
Cleopatra	1	0	0	0	0	0
mercy	1	0	1	1	1	1
worser	1	0	1	1	1	0

Mỗi tài liệu được biểu diễn bằng một vector nhị phân $\in \{0, 1\}$

Ma trận số lần xuất hiện của thuật ngữ – tài liệu

- Hãy xét **số lần xuất hiện của một thuật ngữ trong một tài liệu.**
  - Mỗi tài liệu được biểu diễn như **một vector đếm trong $\mathbb{N}^v$**: một **cột trong bảng bên dưới**.

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	157	73	0	0	0	0
Brutus	4	157	0	1	0	0
Caesar	232	227	0	2	1	1
Calpurnia	0	10	0	0	0	0
Cleopatra	57	0	0	0	0	0
mercy	2	0	3	5	5	1
worser	2	0	1	1	1	0

Mô hình Bag of Words (Túi từ)

- Biểu diễn bằng vector **không xem xét thứ tự của các từ trong một tài liệu**.
- Hai câu:
  - *John is quicker than Mary and Mary is quicker than John*

  sẽ **có cùng vector biểu diễn**.
- Điều này được gọi là mô hình Bag of Words.
- Ở một khía cạnh nào đó, đây là **một bước lùi**: **chỉ mục vị trí** trước đó **có thể phân biệt hai tài liệu này**.

Tần suất thuật ngữ tf

- **Tần suất thuật ngữ** $tf_{t,d}$ của thuật ngữ **t** trong tài liệu **d** được **định nghĩa là số lần mà t xuất hiện trong d**.
  - **Lưu ý:** Trong IR, **tần suất có nghĩa là số lần đếm.**
- Chúng ta muốn **sử dụng tf khi tính điểm mức độ khớp giữa truy vấn và tài liệu**. Nhưng **bằng cách nào?**
- **Tần suất thô** không phải là điều chúng ta muốn.
  - Một tài liệu có **10 lần xuất hiện của thuật ngữ** sẽ **liên quan hơn** một tài liệu có **1 lần xuất hiện** của thuật ngữ đó.
  - Nhưng **không phải là liên quan gấp 10 lần**.
- Mức độ liên quan **không tăng tỉ lệ thuận** với tần suất của thuật ngữ.

Trọng số tần suất dạng log

- Trọng số tần suất dạng log của một từ $t$trong tài liệu d là:

$$w_{t,d} = \begin{cases} 1 + \log_{10} \mathrm{tf}_{t,d}, & \text{if } \mathrm{tf}_{t,d} > 0 \\ 0, & \text{otherwise} \end{cases}$$

- Ví dụ: 0 → 0, 1 → 1, 2 → 1.3, 10 → 2, 1000 → 4, …
- **Điểm số cho một cặp tài liệu – truy vấn** là **tổng trên các từ** t xuất hiện **trong cả truy vấn** q **và tài liệu** d:

$$score = \sum_{t \in q \cap d} \left(1 + \log_{10}\left(tf_{t,d}\right)\right)$$

- Score sẽ bằng 0 nếu không có từ nào của truy vấn xuất hiện trong tài liệu.

Các từ hiếm mang nhiều thông tin hơn

- Các từ hiếm mang nhiều thông tin hơn các từ xuất hiện thường xuyên
  - stop words.
- **Xét một từ trong truy vấn mà rất hiếm trong toàn bộ tập tài liệu** (ví dụ: *arachnocentric*).
- **Một tài liệu chứa từ này rất có khả năng liên quan đến truy vấn** *arachnocentric*.
- **Vì vậy chúng ta muốn gán trọng số cao cho các từ hiếm** như *arachnocentric*.

Tần suất trong bộ sưu tập vs. Tần suất trong tài liệu

- **Collection frequency của t** là **số lần xuất hiện của t trong toàn bộ bộ sưu tập**
- **Document frequency của t** là **số lượng tài liệu mà trong đó t xuất hiện**.

Word	Collection frequency	Document frequency
insurance	10440	3997
try	10422	8760

- Từ nào **tốt hơn cho tìm kiếm** (được **gán trọng số cao hơn**)?

Trọng số idf

- df_t là **tần suất tài liệu (document frequency)** của từ t: **số lượng tài liệu chứa từ** t.
  - df_t là **một thước đo nghịch đảo của mức độ thông tin của từ** t.
  - $df_t < N$
- **Chúng ta định nghĩa idf (inverse document frequency) của** t:

$$idf_t = \log_{10}\left(\frac{N}{df_t}\right)$$

  - Chúng ta sử dụng $\log(N/df_t)$ thay vì N/df_t để **"giảm bớt" (làm dịu) ảnh hưởng của idf**.

Ví dụ

- Tập tài liệu có N = 1 triệu

term	df_t	idf_t
calpurnia	1	6
animal	100	4
sunday	1,000	3
fly	10,000	2
under	100,000	1
the	1,000,000	0

$$\mathrm{idf}_t = \log_{10}(N/\mathrm{df}_t)$$

- Mỗi thuật ngữ t trong một bộ sưu tập có một giá trị idf.

Ảnh hưởng của idf lên xếp hạng

- Liệu idf có ảnh hưởng đến việc xếp hạng đối với các truy vấn **một từ** không, ví dụ như:
  - iPhone
- **idf không ảnh hưởng đến việc xếp hạng** đối với các truy vấn chỉ có **một từ**.
  - **idf ảnh hưởng đến việc xếp hạng tài liệu** đối với các truy vấn có **ít nhất hai từ**.
- Ví dụ: với truy vấn **"capricious person"**, việc dùng **trọng số idf** khiến cho các lần xuất hiện của từ **"capricious"** được tính **quan trọng hơn nhiều** trong xếp hạng tài liệu cuối cùng so với các lần xuất hiện của từ **"person"**.

Trọng số tf-idf

- Trọng số **tf-idf** của một thuật ngữ là tích của **trọng số tf** và **trọng số idf** của nó.
- Là **phương pháp gán trọng số nổi tiếng nhất** trong truy xuất thông tin.
  - Lưu ý: dấu "-" trong **tf-idf** là **dấu gạch nối**, không phải dấu trừ!
  - Tên gọi khác: **tf.idf**, **tf × idf**
- Tăng lên khi số lần xuất hiện của thuật ngữ trong một tài liệu tăng
- Tăng lên khi thuật ngữ đó hiếm hơn trong toàn bộ tập tài liệu
- Công thức:

$$w_{t,d} = \left(1 + \log_{10}\left(tf_{t,d}\right)\right) \times \log_{10}\left(\frac{N}{df_t}\right)$$

Điểm số của một tài liệu khi cho trước một truy vấn

$$\text{Score}(q,d) = \sum_{t \in q \cap d} \text{tf.idf}_{t,d}$$

- Có nhiều biến thể
  - Cách tính **"tf"** (có dùng log hoặc không dùng log)
  - Các **term trong truy vấn** có được gán trọng số hay không
  - ...

Ma trận nhị phân → số đếm → trọng số

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	5.25	3.18	0	0	0	0.35
Brutus	1.21	6.1	0	1	0	0
Caesar	8.59	2.54	0	1.51	0.25	0
Calpurnia	0	1.54	0	0	0	0
Cleopatra	2.85	0	0	0	0	0
mercy	1.51	0	1.9	0.12	5.25	0.88
worser	1.37	0	0.11	4.15	0.25	1.95

Mỗi tài liệu bây giờ được biểu diễn bằng một vector giá trị thực của các trọng số tf-idf ∈ ℝ^M!

Tài liệu dưới dạng vector

- Giả sử chúng ta có một **không gian vector |V|-chiều**
- **Các term** là **các trục của không gian**
- **Các tài liệu** là **các điểm hoặc các vector** trong không gian này
- **Số chiều rất lớn**: hàng **chục triệu chiều** khi áp dụng cho **công cụ tìm kiếm web**
- Đây là **các vector rất thưa – phần lớn các phần tử bằng 0**.

Truy vấn dưới dạng vector

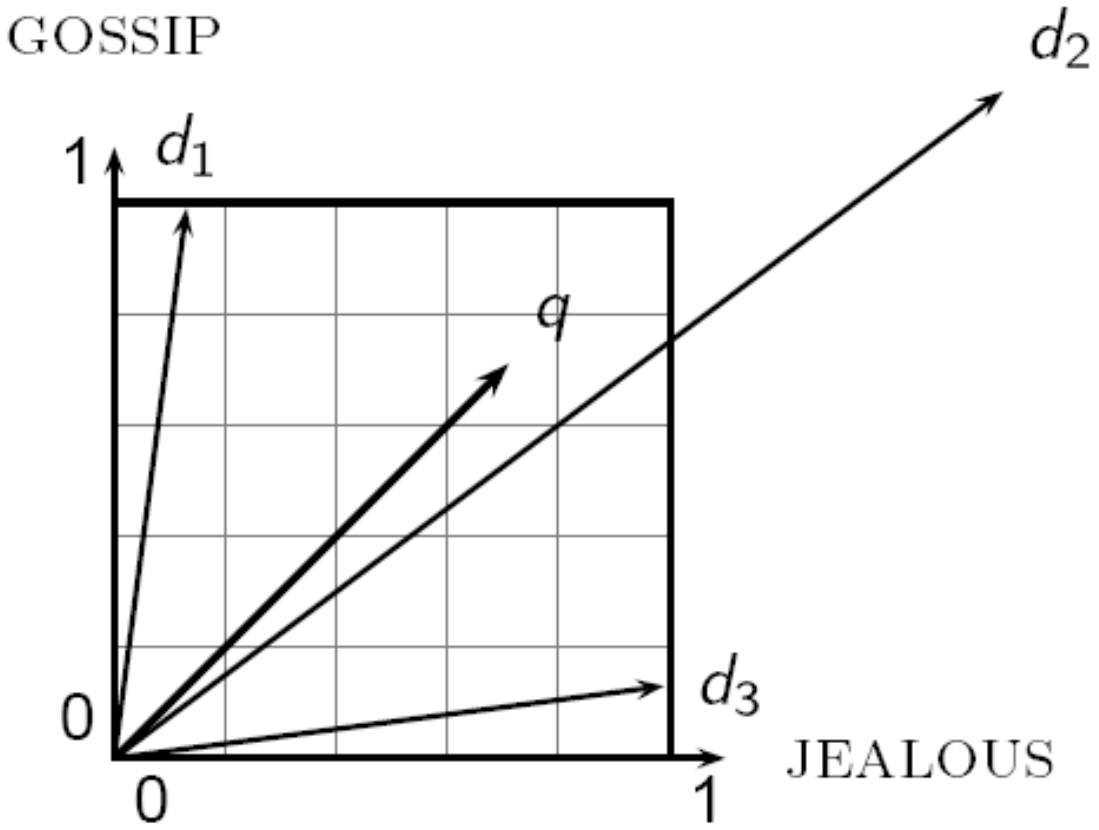
- **Ý tưởng chính 1:** Làm điều tương tự với **truy vấn**: biểu diễn chúng dưới dạng **vector trong không gian**.
- **Ý tưởng chính 2: Xếp hạng tài liệu** dựa theo **mức độ gần với truy vấn trong không gian này**.
- **Độ gần = độ tương tự của các vector**
- **Độ gần = nghịch đảo của khoảng cách**.

Hình thức hóa độ gần trong không gian vector

- Bước đầu: **khoảng cách giữa hai điểm**
  - (= khoảng cách giữa **hai điểm cuối của hai vector**)
- Có nên dùng khoảng cách Euclid?
- Khoảng cách Euclid là một ý tưởng tồi...
  - ... vì **khoảng cách Euclid sẽ lớn đối với các vector có độ dài khác nhau.**

Vì sao khoảng cách là một ý tưởng không tốt

- Khoảng cách **Euclid** giữa **q** và **d_2** là lớn mặc dù **phân bố các term trong truy vấn q** và **phân bố các term trong tài liệu d_2** rất giống nhau.



Sử dụng góc thay vì khoảng cách

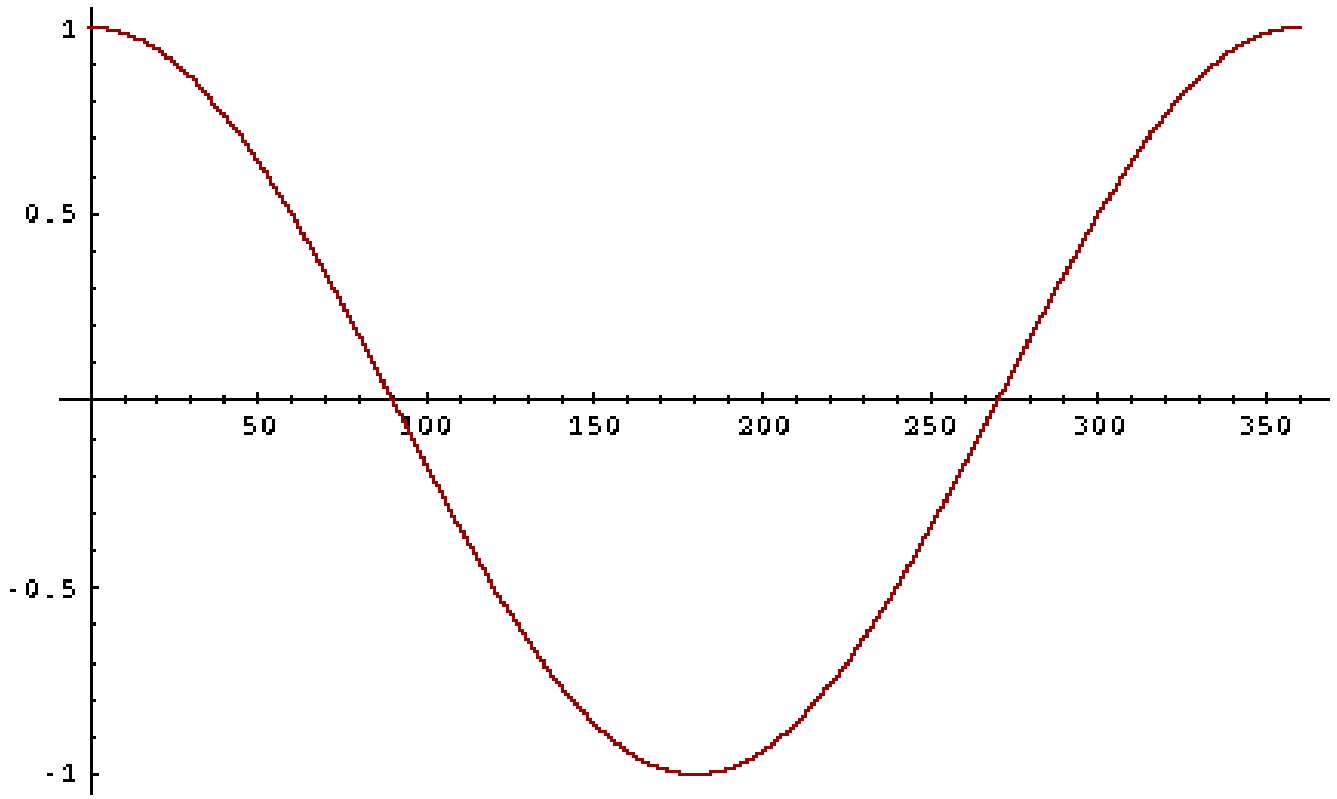
- **Thí nghiệm suy nghĩ:** lấy một tài liệu *d* và nối nó với chính nó. Gọi tài liệu mới là *d′*.
- Về mặt **"ngữ nghĩa"**, *d* và *d′* có cùng nội dung.
- **Khoảng cách Euclid** giữa hai tài liệu có thể khá lớn.
- **Góc giữa hai tài liệu bằng 0**, tương ứng với **độ tương tự tối đa**.
- **Ý tưởng chính:** Xếp hạng các tài liệu theo **góc giữa tài liệu và truy vấn**.

Từ góc đến cosin

- Hai khái niệm sau là **tương đương**.
  - **Xếp hạng tài liệu theo thứ tự giảm dần của góc giữa truy vấn và tài liệu**
  - **Xếp hạng tài liệu theo thứ tự tăng dần của cosine(query, document)**
- Cosine là một hàm giảm đơn điệu trên khoảng [0°, 180°]

Từ góc đến cosin

- Nhưng chúng ta nên tính **cosin** như thế nào?



Chuẩn hoá độ dài

- Một vector có thể được **chuẩn hoá (theo độ dài)** bằng cách chia **mỗi thành phần của nó cho độ dài của vector** – để làm điều này ta dùng **chuẩn L2**:

$$\|\vec{x}\|_2 = \sqrt{\sum_i x_i^2}$$

- Chia một vector cho **chuẩn L2 của nó** sẽ biến nó thành **vector đơn vị** (vector có độ dài bằng 1, nằm trên bề mặt của **siêu cầu đơn vị**).
- Ảnh hưởng đối với hai tài liệu **d và d'** (trong đó **d được nối thêm với chính nó**) từ slide trước: **sau khi chuẩn hoá độ dài, chúng có các vector giống hệt nhau.**
  - Các tài liệu **dài và ngắn** giờ đây có **trọng số có thể so sánh được**.

cosine(query,document)

Dot product

Unit vectors

$$\cos(\vec{q},\vec{d}) = \frac{\vec{q} \bullet \vec{d}}{|\vec{q}||\vec{d}|} = \frac{\vec{q}}{|\vec{q}|} \bullet \frac{\vec{d}}{|\vec{d}|} = \frac{\sum_{i=1}^{|V|} q_i d_i}{\sqrt{\sum_{i=1}^{|V|} q_i^2}\sqrt{\sum_{i=1}^{|V|} d_i^2}}$$

- Dot product - tích vô hướng
- Unit vectors - vector đơn vị
- q_i là **trọng số của term i trong truy vấn**
- d_i là **trọng số của term i trong tài liệu**
- cos(q,d) là **độ tương tự cosine giữa q và d**, hay nói cách khác, **cosine của góc giữa vector q và vector d**.

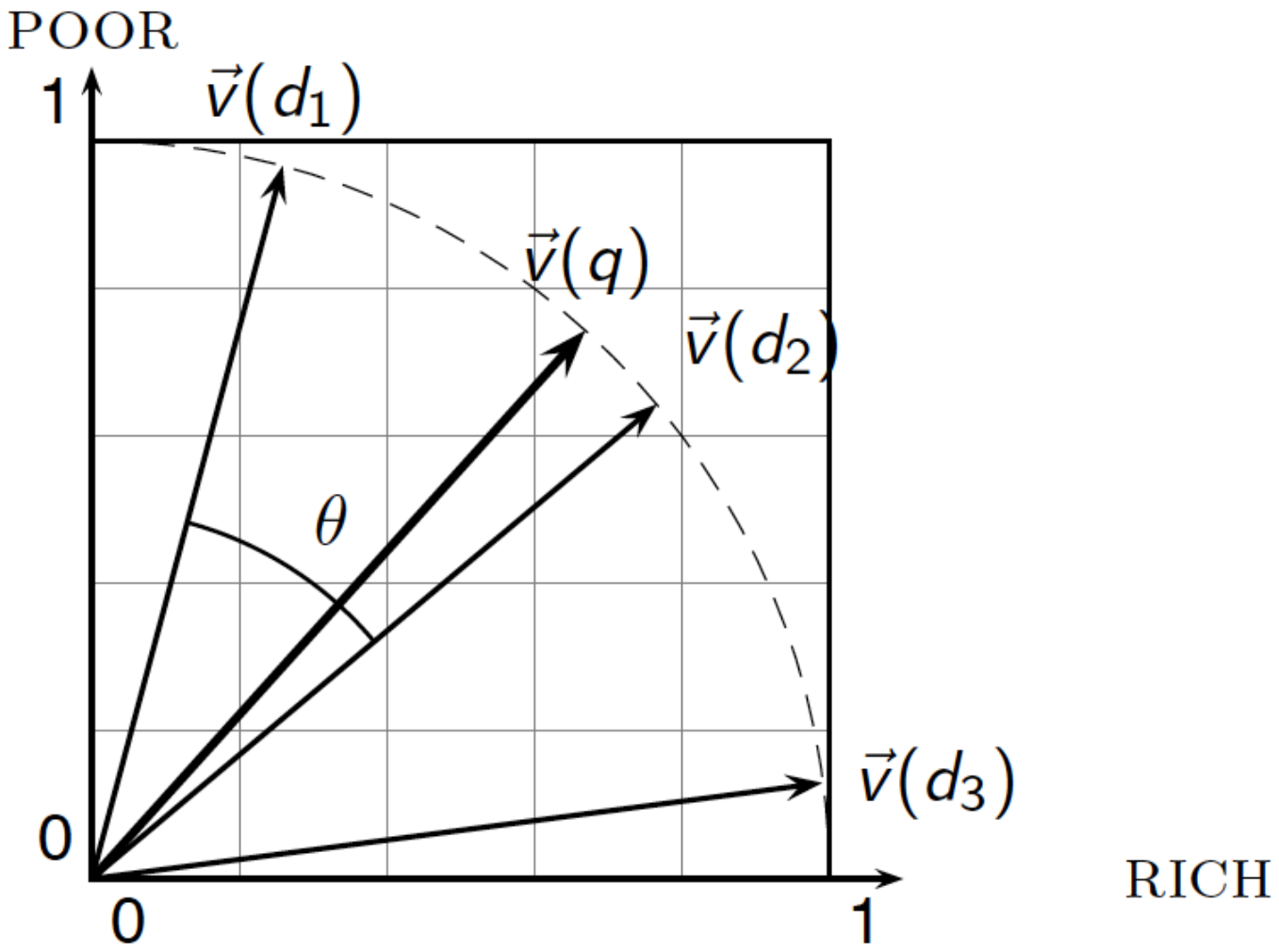
Cosine cho các vector đã được chuẩn hóa độ dài

- Đối với các **vector đã được chuẩn hóa độ dài, độ tương tự cosine** đơn giản chính là **tích vô hướng (dot product hoặc scalar product)**:

$$\cos(\vec{q},\vec{d}) = \vec{q} \bullet \vec{d} = \sum_{i=1}^{|V|} q_i d_i$$

với q, d đã được chuẩn hóa độ dài.

Minh họa độ tương tự Cosine



Độ tương tự cosine giữa 3 tài liệu

- Ba tài liệu sau giống nhau đến mức nào?
  - SaS: *Sense and Sensibility*
  - PaP: *Pride and Prejudice*
  - WH: *Wuthering Heights*
- **Ghi chú:** Để đơn giản hóa ví dụ này, chúng ta **không áp dụng trọng số idf**.

term	SaS	PaP	WH
affection	115	58	20
jealous	10	7	11
gossip	2	0	6
wuthering	0	0	38

Term frequencies (counts)

Ví dụ 3 tài liệu

Trọng số tần suất log

term	SaS	PaP	WH
affection	3.06	2.76	2.30
jealous	2.00	1.85	2.04
gossip	1.30	0	1.78
wuthering	0	0	2.58

Sau khi chuẩn hóa độ dài

term	SaS	PaP	WH
affection	0.789	0.832	0.524
jealous	0.515	0.555	0.465
gossip	0.335	0	0.405
wuthering	0	0	0.588

dot(SaS,PaP) ≈ 12.1
dot(SaS,WH) ≈ 13.4
dot(PaP,WH) ≈ 10.1

cos(SaS,PaP) ≈ 0.94
cos(SaS,WH) ≈ 0.79
cos(PaP,WH) ≈ 0.69

Tính điểm cosine

```
CosineScore(q)
  float Scores[N] = 0
  float Length[N]
  for each query term t
  do calculate w_{t,q} and fetch postings list for t
      for each pair(d, tf_{t,d}) in postings list
      do Scores[d] += w_{t,d} × w_{t,q}
  Read the array Length
  for each d
  do Scores[d] = Scores[d]/Length[d]
  return Top K components of Scores[]
```

Tính điểm cosine

- Thuật toán trước đó tính điểm **theo từng term một (TAAT – term-at-a-time)**
- Thuật toán có thể được điều chỉnh để tính điểm **theo từng tài liệu một (DAAT – document-at-a-time)**
- Việc lưu $\mathbf{w_{t,n}}$ **trong mỗi posting** có thể tốn kém
  - ... vì chúng ta sẽ phải lưu **một số thực dấu phẩy động**
  - Đối với **tf-idf scoring**, chỉ cần lưu $\mathbf{tf_t}$ **trong posting** và $\mathbf{idf_t}$ **trong phần đầu của danh sách posting**
- Việc **trích xuất K kết quả đứng đầu** có thể thực hiện bằng **hàng đợi ưu tiên (priority queue, ví dụ: heap)**

Các biến thể của trọng số tf-idf

Term frequency		Document frequency		Normalization	
n (natural)	$\mathrm{tf}_{t,d}$	n (no)	1	n (none)	1
l (logarithm)	$1+\log(\mathrm{tf}_{t,d})$	t (idf)	$\log\frac{N}{\mathrm{df}_t}$	c (cosine)	$\frac{1}{\sqrt{w_1^2+w_2^2+\ldots+w_M^2}}$
a (augmented)	$0.5+\frac{0.5\times\mathrm{tf}_{t,d}}{\max_t(\mathrm{tf}_{t,d})}$	p (prob idf)	$\max\{0,\log\frac{N-\mathrm{df}_t}{\mathrm{df}_t}\}$	u (pivoted unique)	$1/u$
b (boolean)	$\begin{cases}1 & \text{if } \mathrm{tf}_{t,d}>0\\ 0 & \text{otherwise}\end{cases}$			b (byte size)	$1/CharLength^{\alpha}$, $\alpha<1$
L (log ave)	$\frac{1+\log(\mathrm{tf}_{t,d})}{1+\log(\mathrm{ave}_{t\in d}(\mathrm{tf}_{t,d}))}$				

Trọng số có thể khác nhau giữa truy vấn và tài liệu

- Nhiều **công cụ tìm kiếm** cho phép sử dụng **các cách gán trọng số khác nhau** cho **truy vấn** và **tài liệu**
- **Ký hiệu SMART (SMART Notation):** dùng để biểu diễn tổ hợp phương pháp được sử dụng trong một hệ thống tìm kiếm, với dạng ký hiệu **ddd.qqq**, sử dụng các chữ viết tắt từ bảng trước đó.
- Một **lược đồ gán trọng số rất phổ biến** là: **lnc.ltc**
- **Tài liệu:** tf dạng **logarithmic** (chữ **l** ở vị trí đầu), **không dùng idf**, và **chuẩn hóa cosine**
- **Truy vấn:** tf dạng **logarithmic** (chữ **l** ở cột bên trái), **idf** (chữ **t** ở cột thứ hai), **chuẩn hóa cosine** …

Ví dụ tf-idf: lnc.ltc

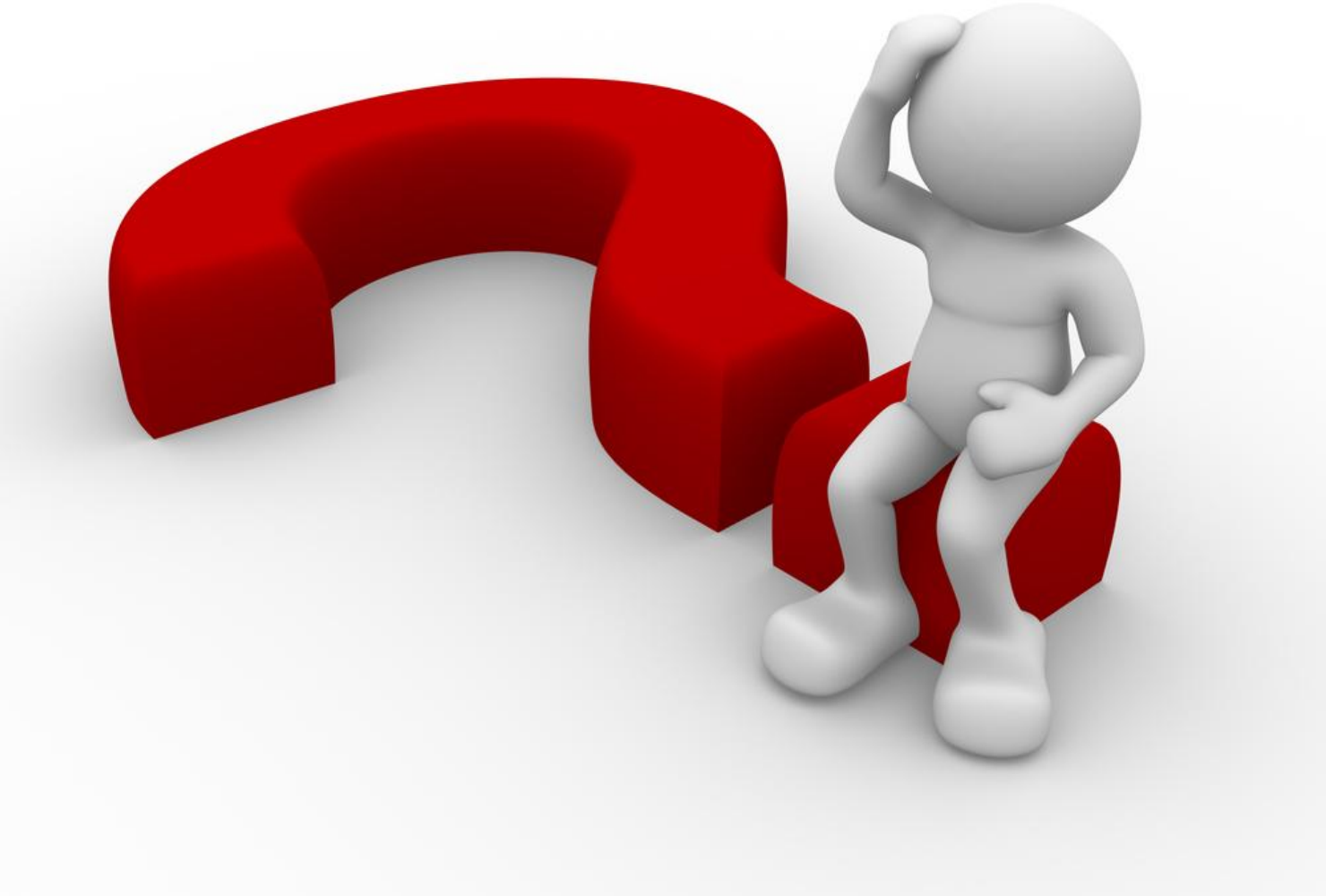
- Tài liệu: **car insurance auto insurance**
- Truy vấn: **best car insurance**

Term	Query						Document				Prod
	tf-raw	tf-wt	df	idf	wt	n'lize	tf-raw	tf-wt	wt	n'lize	
auto	0	0	5000	2.3	0	0	1	1	1	0.52	0
best	1	1	50000	1.3	1.3	0.34	0	0	0	0	0
car	1	1	10000	2.0	2.0	0.52	1	1	1	0.52	0.27
insurance	1	1	1000	3.0	3.0	0.78	2	1.3	1.3	0.68	0.53

Độ dài tài liệu = $\mathbf{1^2 + 0^2 + 1^2 + 1.3^2 \approx 1.92}$

Score = **0 + 0 + 0.27 + 0.53 = 0.8**

Tóm tắt – xếp hạng theo mô hình không gian vector

- Biểu diễn **truy vấn** như một **vector tf-idf có trọng số**
- Biểu diễn **mỗi tài liệu** như một **vector tf-idf có trọng số**
- Tính **điểm tương đồng cosine** giữa **vector truy vấn** và **vector của từng tài liệu**
- Xếp hạng các tài liệu theo **điểm số** so với truy vấn
- Trả về **K tài liệu đứng đầu** (ví dụ: **K = 10**) cho người dùng

Hỏi đáp